

**Swinburne University of Technology**  
*Faculty of Science, Engineering and Technology*

**ASSIGNMENT COVER SHEET**

---

**Subject Code:** COS30008  
**Subject Title:** Data Structures and Patterns  
**Assignment number and title:** 4, Binary Search Trees & In-Order Traversal  
**Due date:** May 26, 2022, 14:30  
**Lecturer:** Dr. Markus Lumpe

---

**Your name:** \_\_\_\_\_ **Your student id:** \_\_\_\_\_

|                   |              |              |               |               |               |               |               |              |              |              |              |
|-------------------|--------------|--------------|---------------|---------------|---------------|---------------|---------------|--------------|--------------|--------------|--------------|
| Check<br>Tutorial | Mon<br>10:30 | Mon<br>14:30 | Tues<br>08:30 | Tues<br>10:30 | Tues<br>12:30 | Tues<br>14:30 | Tues<br>16:30 | Wed<br>08:30 | Wed<br>10:30 | Wed<br>12:30 | Wed<br>14:30 |
|                   |              |              |               |               |               |               |               |              |              |              |              |

---

Marker's comments:

| Problem | Marks   | Obtained |
|---------|---------|----------|
| 1       | 94      |          |
| 2       | 42      |          |
| 3       | 8+86=94 |          |
| Total   | 230     |          |

---

**Extension certification:**

This assignment has been given an extension and is now due on \_\_\_\_\_

Signature of Convener: \_\_\_\_\_

```
1
2 // COS30008, Problem Set 4, Problem 2, 2022
3
4 #pragma once
5
6 // #include "BinarySearchTreeIterator.h"
7 #include "BinaryTreeNode.h"
8
9 #include <stdexcept>
10
11 // Problem 3 requirement
12 template <typename T> class BinarySearchTreeIterator;
13
14 template <typename T> class BinarySearchTree {
15     private:
16         using BNode = BinaryTreeNode<T>;
17         using BTreeNode = BNode *;
18
19         BTreeNode fRoot;
20
21     public:
22         BinarySearchTree()
23             : fRoot(&BNode::NIL) {
24
25         };
26
27         ~BinarySearchTree() {
28             if (!this->fRoot->empty()) {
29                 delete this->fRoot;
30             }
31         };
32
33         bool empty() const { return this->fRoot->empty(); };
34         size_t height() const { return this->fRoot->height(); };
35
36         bool insert(const T &aKey) {
37             if (this->empty()) {
38                 this->fRoot = new BinaryTreeNode<T>(aKey);
39                 return true;
40             }
41             return this->fRoot->insert(aKey);
42         }
43         bool remove(const T &aKey) {
44             if (this->empty()) {
45                 return false;
46             }
47             return this->fRoot->remove(aKey, this->fRoot);
48         }
49 }
```

```
50     // Problem 3 methods
51
52     using Iterator = BinarySearchTreeIterator<T>;
53
54     // Allow iterator to access private member variables
55     friend class BinarySearchTreeIterator<T>;
56
57     Iterator begin() const { return Iterator(*this).begin(); }
58     Iterator end() const { return Iterator(*this).end(); }
59 };
60
```

```
1
2 // COS30008, Problem Set 4, Problem 1, 2022
3
4 #pragma once
5
6 #include <algorithm>
7 #include <cstdint>
8 #include <stdexcept>
9 #include <utility>
10
11 template <typename T> struct BinaryTreeNode {
12     using BNode = BinaryTreeNode<T>;
13     using BTreeNode = BNode *;
14
15     T key;
16     BTreeNode left;
17     BTreeNode right;
18
19     static BNode NIL;
20
21     const T &findMax() const {
22         if (empty()) {
23             throw std::domain_error("Empty tree encountered.");
24         }
25
26         return right->empty() ? key : right->findMax();
27     }
28
29     const T &findMin() const {
30         if (empty()) {
31             throw std::domain_error("Empty tree encountered.");
32         }
33
34         return left->empty() ? key : left->findMin();
35     }
36
37     bool remove(const T &aKey, BTreeNode aParent) {
38         BTreeNode x = this;
39         BTreeNode y = aParent;
40
41         while (!x->empty()) {
42             if (aKey == x->key) {
43                 break;
44             }
45
46             y = x; // new parent
47
48             x = aKey < x->key ? x->left : x->right;
49         }
```

```

50
51     if (x->empty()) {
52         return false; // delete failed
53     }
54
55     if (!x->left->empty()) {
56         const T &lKey = x->left->findMax(); // find max to left
57         x->key = lKey;
58         x->left->remove(lKey, x);
59     } else {
60         if (!x->right->empty()) {
61             const T &lKey = x->right->findMin(); // find min to right
62             x->key = lKey;
63             x->right->remove(lKey, x);
64         } else {
65             if (y != &NIL) // y can be NIL
66             {
67                 if (y->left == x) {
68                     y->left = &NIL;
69                 } else {
70                     y->right = &NIL;
71                 }
72             }
73
74             delete x; // free deleted node
75         }
76     }
77
78     return true;
79 }
80
81 // PS4 starts here
82
83 // is nil supposed to be whatever it is before they invented nullptr?;
84
85 // I love how the type aliasing makes thing MUCH MORE CONFUSING
86
87 BinaryTreeNode() {
88     this->left = &NIL;
89     this->right = &NIL;
90 }
91 BinaryTreeNode(const T &aKey) {
92     // copy constructor?
93     this->key = aKey;
94     this->left = &NIL;
95     this->right = &NIL;
96 };
97 BinaryTreeNode(T &aKey) {
98     this->key = std::move(aKey);

```

```
99     this->left = &NIL;
100     this->right = &NIL;
101 };
102
103 ~BinaryTreeNode() {
104     if (!this->left->empty()) {
105         delete left;
106     }
107     if (!this->right->empty()) {
108         delete right;
109     }
110 };
111
112 bool empty() const { return this == &NIL; };
113 bool leaf() const { return this->left == &NIL && this->right == &NIL; };
114 size_t height() const {
115     if (this->empty()) {
116         throw std::domain_error("Empty tree encountered");
117     } else if (this->leaf()) {
118         return 0;
119     } else {
120         size_t t = 0;
121         if (!this->left->empty()) {
122             t = std::max(t, this->left->height() + 1);
123         }
124         if (!this->right->empty()) {
125             t = std::max(t, this->right->height() + 1);
126         }
127         return t;
128     }
129 };
130
131 bool insert(const T &aKey) {
132     // auto node = new BinaryTreeNode(aKey);
133     if (this->empty()) {
134         return false;
135     } else {
136         auto node = this;
137         while (!node->empty()) {
138             if (aKey == node->key) {
139                 return false;
140             } else if (aKey < node->key) {
141                 if (node->left->empty()) {
142                     node->left = new BinaryTreeNode<T>(aKey);
143                     return true;
144                 } else {
145                     node = node->left;
146                 }
147             }
148         }
149     }
150 }
```

```
147         } else if (aKey > node->key) {
148             if (node->right->empty()) {
149                 node->right = new BinaryTreeNode<T>(aKey);
150                 return true;
151             } else {
152                 node = node->right;
153             }
154         }
155     }
156     return false;
157 }
158 };
159 };
160
161 template <typename T> BinaryTreeNode<T> BinaryTreeNode<T>::NIL;
162
```

```
1
2 // COS30008, Problem Set 4, Problem 3, 2022
3
4 #pragma once
5
6 #include "BinarySearchTree.h"
7
8 #include <exception>
9 #include <stack>
10 #include <stdexcept>
11
12 template <typename T> class BinarySearchTreeIterator {
13     private:
14         using BSTree = BinarySearchTree<T>;
15         using BNode = BinaryTreeNode<T>;
16         using BTreeNode = BNode *;
17         using BTNStack = std::stack<BTreeNode>;
18
19         const BSTree &fBSTree; // binary search tree
20         BTNStack fStack;       // DFS traversal stack
21
22         void pushLeft(BNode *aNode) {
23             while (!aNode->empty()) {
24                 this->fStack.push(aNode);
25                 aNode = aNode->left;
26             }
27         };
28
29     public:
30         using Iterator = BinarySearchTreeIterator<T>;
31
32         BinarySearchTreeIterator(const BinarySearchTree<T> &aBSTree)
33             : fBSTree(aBSTree) {}
34
35         const T &operator*() const {
36             if (this->fStack.empty()) {
37                 throw std::out_of_range("Empty root");
38             }
39             return this->fStack.top()->key;
40         }
41
42         Iterator &operator++() {
43             // if node has a left node, go into that node
44
45             if (!this->fStack.empty()) {
46                 // travel root?
47                 BTreeNode current_root = this->fStack.top();
48                 this->fStack.pop();
49
```



```
50         if (!current_root->right->empty()) {
51             this->pushLeft(current_root->right);
52         }
53     }
54
55     return *this;
56 };
57
58 Iterator operator++(int) {
59     Iterator old = *this;
60     ++(*this);
61     return old;
62 }
63 bool operator==(const Iterator &aOtherIter) const {
64     // check if two stacks are equal maybe?
65     return ((this->fStack.empty() && aOtherIter.fStack.empty()) ||
66            (this->fStack.size() == aOtherIter.fStack.size() &&
67             this->fStack.top()->key == aOtherIter.fStack.top()->key));
68 };
69 bool operator!=(const Iterator &aOtherIter) const {
70     return !(*this == aOtherIter);
71 };
72
73 Iterator begin() const {
74     // YOU CAN JUST DO THIS?
75     Iterator it = Iterator(this->fBSTree);
76     if (!this->fBSTree.empty()) {
77         it.pushLeft(this->fBSTree.fRoot);
78     }
79     return it;
80 };
81 Iterator end() const { return Iterator(this->fBSTree); };
82 };
83
```